pfi

Python for Fixed Income

COURSE 9 · DEPLOYMENT

FastAPI: A Bond Math Service

WEEK 2 · DAY 1

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Today: a bond math service

Today

1. What a service is: request, response, status
2. The request and response models
3. bond_app: four endpoints and a schema
4. The test client: no port, no process
5. A bad request: a 422 naming the field
6. One real server, started and stopped
7. What FastAPI does not check
8. Five bonds: the service, bondmath, Excel

Three packages

`fastapi`

the web framework

`pydantic`

checks each request

`uvicorn`

the server program

Today the service runs on your machine. Day 7 puts it at a public URL.

What the service returns, and what it never says

Numbers in, numbers out	units in every key: price_per_100, dv01_per_100
Synthetic bonds	the week's bond and five bonds of the synthetic book
It computes	it holds no data and never says a number is high or low

The service's outputs

Allowed names

price per 100 of face

DV01 per 100 of face

the service returned

Never called

fair value, cheap, or rich

a quote or a mark

a recommendation

What a service is

On your laptop

```
bondmath.dv01(
  dates, 5.0, 5.06, 2)
0.043625
```


Through a service

```
POST /dv01
{"settlement": "2026-10-02", "maturity": "2031-10-02", ...}
status 200  {"dv01_per_100": 0.043624879198739 52}
```

method	GET reads; POST sends a body
path	/dv01: which function
body	JSON text: the arguments
status	200 success; 422 refused

The request model

```
class BondRequest(pydantic.BaseModel):
    settlement: date
    maturity: date
    coupon_pct: float  (strict)
    yield_pct: float  (strict)
    frequency: Literal[1, 2, 4]
```

Three rules

Every field required: no default

Numbers strict: "5.06" refused

Maturity after settlement

```
refused: 1 problem; field ('yield_pct',), missing: Field required
```

Response models hold one number each, unit in the name: price_per_100, dv01_per_100.

bond_app: routes and a schema

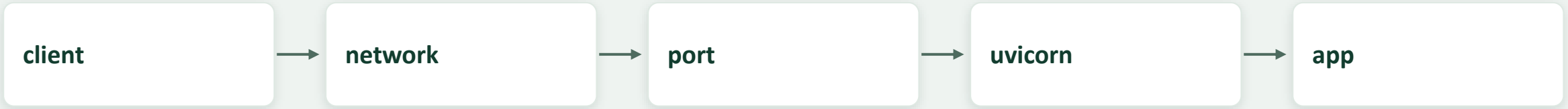
```
GET   /health   Health
POST  /price    Price
POST  /accrued  Accrued
POST  /dv01     Dv01
POST  /yield    Yield To Maturity
```

```
required: ['settlement', 'maturity', 'coupon_pct', 'yield_pct', 'frequency']
settlement  {'type': 'string', 'format': 'date'}
maturity  {'type': 'string', 'format': 'date'}
coupon_pct  {'type': 'number'}
yield_pct  {'type': 'number'}
frequency  {'type': 'integer', 'enum': [1, 2, 4]}
```

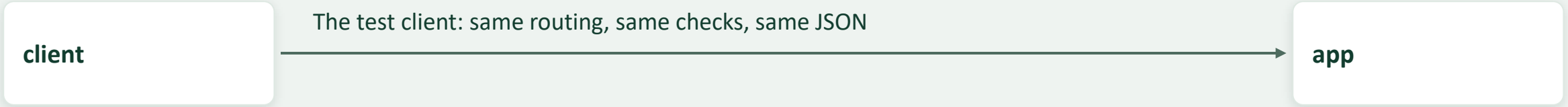
FastAPI writes the schema from your models; a running service shows it at /docs.

GET /health: {"status": "ok", ...}; as_of None, the service holds no data

The test client: no port, no process



A real server




```
the service: 0.04362487919873952
bondmath:    0.04362487919873952
```

Equal to the last digit. 22 tests pass, none opens a port.

A bad request is a 422 that names the field

```
the yield left out: status 422; field ['body', 'yield_pct']
the coupon as text: status 422; field ['body', 'coupon_pct']
settlement after maturity: status 422; field ['body', 'maturity']
```

`Field required`

`Input should be a valid number`

`Value error, maturity must be after settlement`

bondmath never ran. The 422 body echoes the input: one reason a request log never stores bodies.

One real server, started and stopped in one cell

port held? refuse → start uvicorn → wait for /health → hand over the URL → stop; port free

```
the server answers at http://127.0.0.1:8765
GET /health: status 200, {'status': 'ok', 'version': '1.0', 'as_of': None}
POST /dv01: status 200, {'dv01_per_100': 0.04362487919873952}
equal to bondmath.dv01 to the last digit: True
a second server: refused, port 8765 is already in use on 127.0.0.1
port 8765 free after the block: True
the server's log: 0 bytes
```

127.0.0.1 is this machine only. The rest of the course uses the test client.

What FastAPI does not check

```
yield_pct: 506.0   (meant 5.06)
status 200; price_per_100 0.988471856785912
```

the shape

fields, types, dates: the model

the numbers

a yield's sense: your tests

FastAPI 0.143.0 (2026-10-08): automatic OpenTelemetry setup is opt-in. The course never switches it on.

Five bonds, the service, and Excel

cusip	maturity	coupon	yield	price/100	DV01/100	= bondmath
99080CAE8	2032-06-18	7.000	5.606	106.726595	0.050300	True
99049XAE2	2029-04-17	8.125	9.220	97.555694	0.021915	True
99039NAG1	2053-04-09	6.125	6.756	92.260704	0.115277	True
99034IAA0	2038-09-17	4.625	5.960	88.692370	0.078608	True
99127XAB3	2029-06-26	5.250	4.590	101.673195	0.025742	True

```
A2  =PRICE(DATE(2026,9,30),DATE(2032,6,18),0.07,0.05606,100,2,0)
B2  =100*0.07/2*COUPDAYBS(...)/COUPDAYS(...)
C2  =MDURATION(DATE(2026,9,30),DATE(2032,6,18),0.07,0.05606,2,0)*(A2+B2)/10000
```

Excel: prices within 3e-14, DV01s within 3e-17; equal at 6 decimals, all six bonds.

Excel and Git

WEBSERVICE on the running service

```
/health: {"status":"ok","version":"1.0","as_of":null}

/dv01: #VALUE!   WEBSERVICE sends GET only; /dv01 takes POST
```

```
deploykit day 6: the bond service (bond_app, serve) and service.py, 25 tests
```

```
.gitignore:5:uvicorn_*.log   uvicorn_8765.log
```

Six files in one commit; the holdings file and the server's log are ignored.

The AI check

The docstring is the prompt. The deploy code that comes back runs and looks finished.

1

Docstring

Units, conventions, and what the code must never let through: what you asked for.

2

Code back

Written for the course in the style of an AI answer. Read it.

3

Test first

Plant the case the docstring promises to catch. Run it: it fails.

4

Fix

Find the rule it broke, fix it, run the test again.

Today: yield_pct: float = 5.0, a default: a body that forgot the yield is priced at 5% with status 200.

Code that runs is not the same as code that is right.

What you can do now

- A service: method, path, body, status, JSON
- Models: every field required, units in names
- Test every endpoint with the test client
- Read a 422 and the field it names
- Start and stop one real server with serve

Week 2, Day 2 Deploying the service to Vercel